

CyberAware — Development Walkthrough

Password Strength Analyzer & Phishing Email Scanner: detection logic and UI wiring

Project	CyberAware — Interactive Cybersecurity Awareness Platform
Scope of this document	Build Order steps 2 & 3 (Section 5 of PLAN.md): Phishing Email Scanner, Password Strength Analyzer
Stack	Astro (static site) + vanilla TypeScript/JavaScript, Tailwind CSS. No backend, no external API, no network calls.
Key files	<code>src/lib/password-analyzer.js</code> , <code>src/lib/phishing-detector.js</code> , <code>src/pages/tools/password-strength.astro</code> , <code>src/pages/tools/phishing-scanner.astro</code>

Contents

- 1. Architecture note: no external API
- 2. Password Strength Analyzer — detection logic
- 3. Phishing Email Scanner — detection logic
- 4. The shared pattern across both tools
- 5. UI & wiring — how the .astro pages call the logic and render results

1. Architecture note: no external API

Neither tool uses an external API. Everything runs as custom heuristic logic written from scratch, entirely client-side — a deliberate architecture decision (the project commits to "no backend, no network calls, no data persistence"). There is no call to any AI service, no threat-intelligence feed, nothing leaves the browser. The rest of this document explains what is actually doing the "detection" instead: hand-written pattern matching, a dictionary-normalization technique, and one real algorithm (Levenshtein edit distance).

2. Password Strength Analyzer — `src/lib/password-analyzer.js`

The core idea: this isn't one algorithm, it's a small pipeline of independent checks that each look for one specific weakness, plus a scoring formula that turns those findings into a 0–100 number and a 5-tier label (Very Weak → Very Strong).

The checks themselves

Each is a plain function, easy to test in isolation:

- **Common password list** (lines 13–24) — a `Set` of ~48 real leaked/guessed passwords (`123456` , `password1` , etc). `Set.has()` is $O(1)$, so checking against it is instant regardless of list size.
- **Keyboard walks** (lines 35–38, used at line 81) — literal substring matching for things like `qwerty` or `1qaz2wsx` , which follow the physical layout of a keyboard rather than being random.
- **Sequential runs** (lines 51–64) — actual logic, not a lookup list. It slides a window across the password comparing character codes (`charCodeAt`) to detect 4+ characters in a row that increase or decrease by exactly 1 — catches `abcd` , `4321` , `wxyz` , anything sequential, without needing to enumerate every possible sequence.
- **Repeated runs** (lines 66–79) — same sliding-window idea, checking for the same character repeated 3+ times (`aaa` , `111`).
- **Disguised dictionary words** (lines 40–49 + 86–89) — the most interesting check. A lot of people think `P@ssw0rd` is clever because it "looks" complex. The `normalizeLeetspeak()` function reverses common substitutions (`@` → `a` , `0` → `o` , `3` → `e` , `1 / !` → `i` , `$` → `s` , `7` → `t`) before checking against a base-word list, so `P@ssw0rd` normalizes back to `password` and gets caught anyway — because real cracking tools do exactly this.

```
function normalizeLeetspeak(value) {
  return value
    .toLowerCase()
    .replace(/[@4]/g, "a")
    .replace(/[/3]/g, "e")
    .replace(/[/1!|]/g, "i")
    .replace(/[/0]/g, "o")
    .replace(/[/5]/g, "s")
    .replace(/[/7]/g, "t");
}

function hasSequentialRun(password, minLength = 4) {
  const lower = password.toLowerCase();
  for (let i = 0; i <= lower.length - minLength; i++) {
    let ascending = true;
    let descending = true;
    for (let j = 1; j < minLength; j++) {
      const diff = lower.charCodeAt(i + j) - lower.charCodeAt(i + j - 1);
      if (diff !== 1) ascending = false;
      if (diff !== -1) descending = false;
    }
    if (ascending || descending) return lower.slice(i, i + minLength);
  }
  return null;
}
```

Scoring

Starts from length (up to 20 chars × 4 points = 80) plus a bonus per character class used (uppercase / lowercase / digit / symbol), then subtracts penalties for each weakness found, clamped to 0–100.

```
let score = Math.min(password.length, 20) * 4;
score += Math.max(0, classCount - 1) * 10;

if (isCommonPassword) score = Math.min(score, 5);
if (isAllDigits) score = Math.min(score, 20);
if (weakBaseWord) score -= 25;
if (keyboardWalk) score -= 20;
if (sequentialRun) score -= 15;
if (repeatedRun) score -= 15;
if (password.length < 8) score = Math.min(score, 25);

score = Math.max(0, Math.min(100, Math.round(score)));
```

Crack-time estimate

This is where a real bug was found and fixed during development. The first version used pure brute-force math ($\text{charsetSize}^{\text{length}} \div \text{guesses-per-second}$) for every password, and it gave `qwertyuiop` a

reassuring "2 hours" — technically true for random guessing, but wrong in practice, because real attackers try dictionary + pattern lists *before* brute force. Fixed so keyboard-walk and disguised-word passwords report "seconds to minutes" instead of the misleading math-only estimate.

```
// Real-world cracking tools run dictionary + mutation-rule attacks before
// falling back to brute force, so a keyboard walk or a disguised common
// word is guessed almost immediately.
let crackTime;
if (isCommonPassword) {
  crackTime = "instantly (it's a known common password)";
} else if (keyboardWalk || weakBaseWord) {
  crackTime = "seconds to minutes (it follows a well-known pattern)";
} else {
  crackTime = estimateCrackTime(password, charsetSize);
}
```

UI side

In `src/pages/tools/password-strength.astro`, the input has an `oninput` listener that calls `analyzePassword()` on every keystroke — cheap enough to run live since it's pure computation, no network round-trip. There's a screen-reader-only live region that only announces when the strength *tier* actually changes (not every keystroke, which would be unusable with a screen reader).

3. Phishing Email Scanner — `src/lib/phishing-detector.js`

Architecture: the same pattern as the password analyzer — a list of independent check functions (each returning a flag or `null`), combined and scored by one orchestrator, `analyzeEmail()`. This checks a plain object `{ from, replyTo, subject, body }`, not a live DOM form — that's what makes it independently reusable (the module's own docstring notes it could back a quiz question later).

The core check: brand-spoofing detection

This is the "smart" part of the tool. It answers: *does the sender claim to be a real company, but isn't actually sending from that company's domain?*

1. `findMentionedBrand()` scans the sender name and subject line for any of ~12 commonly-impersonated brand names (PayPal, Microsoft, Netflix, DHL, etc).
2. If a brand is mentioned, `domainMatchesBrand()` checks whether the sender's actual email domain matches that brand's *real* domain(s) (also hardcoded).
3. If it doesn't match, `isLookalikeOfBrand()` runs a **Levenshtein edit-distance** calculation — the classic dynamic-programming algorithm for "how many single-character edits turn string A into string B." If the sender's domain is only 1–2 character-edits away from the real brand name (e.g. `paypa1-alerts.com` vs `paypal`), it's flagged as a **lookalike domain** specifically, with a more precise message than a generic mismatch.

```
function levenshtein(a, b) {
  const m = a.length;
  const n = b.length;
  const dp = Array.from({ length: m + 1 }, (_, i) => [i, ...Array(n).fill(0)]);
  for (let j = 0; j <= n; j++) dp[0][j] = j;
  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      dp[i][j] =
        a[i - 1] === b[j - 1]
          ? dp[i - 1][j - 1]
          : 1 + Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]);
    }
  }
  return dp[m][n];
}

function isLookalikeOfBrand(domain, brand) {
  if (!domain) return false;
  const label = registrableLabel(domain);
  const distance = levenshtein(label, brand.replace(/\s+/g, ""));
  return distance > 0 && distance <= 2 && label.length >= brand.length - 2;
}
```

That Levenshtein function is the one piece of "real" algorithm in the whole project (as opposed to string matching/regex) — it's the same technique spell-checkers use, just applied to domain names instead of dictionary words.

Link analysis

- `extractLinks()` pulls out both Markdown-style `[text](url)` links and bare `https://...` URLs via regex, so it handles however the email body is written.
- For each link it checks: is the destination a raw IP address? Is it a known URL shortener? Does the *visible* link text (if it looks like a URL) actually match where it *points*? — that last one catches the classic trick of displaying `paypal.com` as text while linking somewhere else entirely.

Security decision worth highlighting: extracted links are only ever rendered as plain text in the UI — never as real, clickable `<a>` tags. Since this tool might eventually be fed real pasted phishing emails, it should never be possible to accidentally click through to a live malicious URL from inside a security-education tool.

Scoring

Same idea as the password tool — a numeric score for display, but the actual risk *tier* is driven by "is there any single high-severity flag," not just the accumulated total. This was changed after testing showed a single strong signal (like a spoofed domain) was getting diluted to "Suspicious" when it should stand alone as "High risk."

```
// A single high-severity flag (e.g. a spoofed sender domain) is disqualifying
// on its own – risk level is driven by the strongest flag present, not just
// an accumulated score.
const hasHigh = flags.some((flag) => flag.severity === "high");
const hasMedium = flags.some((flag) => flag.severity === "medium");

let riskLevel = "low";
let riskLabel = "Low risk – no strong red flags detected";
if (hasHigh) {
  riskLevel = "high";
  riskLabel = "High risk – likely phishing";
} else if (hasMedium) {
  riskLevel = "medium";
  riskLabel = "Suspicious – proceed with caution";
} else if (flags.length > 0) {
  riskLabel = "Low risk – a few minor signals, but nothing strongly suspicious";
}
```

4. The shared pattern across both tools

Both follow the same three ideas, which is really the "architecture" of this whole project:

1. **Logic lives in `src/lib/` , completely decoupled from the DOM** — either module can run in Node with zero setup, which is exactly how they were tested (known passwords/emails fed through, output checked) before any UI was wired up.
2. **Detection = many small, independent, explainable checks**, not one opaque score. Each check knows how to explain *why* it fired in plain language — the point of a teaching tool, not just "this is 73% risky."
3. **Nothing leaves the browser.** No `fetch()` , no API keys, no telemetry — the "no data collection" promise on the site is true by construction, not by policy, because there is simply no code path that sends data anywhere.

5. UI & wiring — how the .astro pages call the logic and render results

Every `.astro` file has two distinct parts that run at completely different times, and understanding that split is the key to understanding how these pages work.

Part 1: the frontmatter (runs on the server / at build time)

At the top of each file, between the `---` fences, is server-side code — it runs once when the page is built, never in the browser.

```
---
import BaseLayout from "../../layouts/BaseLayout.astro";
import sampleEmails from "../../data/sample-emails.json";
---
```

That JSON import happens at build time — Astro reads `sample-emails.json` and loops over it right there in the template:

```
{sampleEmails.map((sample) => (
  <button type="button" data-sample-id={sample.id}>{sample.label}</button>
))}
```

So the sample buttons aren't created by JavaScript in the browser — they're already baked into the static HTML before it's ever sent. Each button gets a `data-sample-id` attribute, which is the hook the client script uses later to know *which* sample was clicked. Adding a new sample email needs zero JS changes — just add it to the JSON file and the loop produces another button automatically.

Part 2: the `<script>` tag (runs later, in the browser)

Below the template, there's a second, separate block:

```
<script>
import { analyzePassword } from "../../lib/password-analyzer.js";
const input = document.getElementById("password-input") as HTMLInputElement;
...
</script>
```

This is a separate world — it's bundled by Vite into its own JS module and shipped to the browser as a `<script type="module">`. It imports the lib function fresh (a second import, independent from the frontmatter), and it's written in TypeScript (`as HTMLInputElement` etc.) purely for editor safety — Astro

strips all the type annotations at build time, so what actually ships to the browser is plain JavaScript (verified directly by inspecting the compiled output during testing).

Every interactive element in the HTML has a plain `id` (`password-input` , `strength-label` , `flags-list` , etc.), and the script's first job is just grabbing references to all of them with `document.getElementById()` . That's the entire "framework" here — no React, no Vue, just direct DOM manipulation, matching the project's tech-stack decision: vanilla JS per tool, no framework dependency.

The wiring loop (same shape in both tools)

1. **User does something** — types in the password field, submits the email form, or clicks a sample button.
2. **Gather plain data** from the form fields into a plain object — e.g. builds `{ from, replyTo, subject, body }` straight from input `.value` s.
3. **Call the lib function** — `analyzeEmail(email)` or `analyzePassword(input.value)` . This is the *only* place the UI touches the detection logic — one function call, one plain object back.
4. **Turn the result into HTML strings** and dump them into a container via `.innerHTML` . `renderResults()` maps over `result.flags` and builds an `<li>` per flag with a template literal.
5. **Unhide the results** (`classList.remove("hidden")`) and, for the phishing scanner, scroll to them — but only smoothly if the user doesn't have `prefers-reduced-motion` set.

```
form.addEventListener("submit", (event) => {
  event.preventDefault();

  const email = {
    from: { name: fields.fromName.value.trim(), email: fields.fromEmail.value.trim() },
    replyTo: fields.replyTo.value.trim() || undefined,
    subject: fields.subject.value.trim(),
    body: fields.body.value,
  };

  const result = analyzeEmail(email);
  renderResults(result);
});
```

The one security-relevant detail in the wiring: `escapeHtml()`

Both scripts define the exact same tiny helper:

```
function escapeHtml(value: string) {
  const div = document.createElement("div");
  div.textContent = value;
  return div.innerHTML;
}
```

This matters because the results are built with `.innerHTML = `...${flag.title}...``, and `flag.title` can ultimately trace back to user input (e.g. the email body someone pasted, echoed back inside an evidence string). Setting `.textContent` on a throwaway `<div>` and reading back `.innerHTML` forces the browser to HTML-escape it (`<` becomes `<`, etc.) — a cheap, dependency-free way to avoid building a self-XSS hole out of a security-education tool. Every piece of dynamic text that goes into the rendered HTML is run through this first; only hardcoded `class="..."` strings and static markup skip it.

Password analyzer's one difference: it's live, not submit-based

The phishing scanner and URL checker both wait for a form `submit`. The password analyzer runs on every keystroke (`input` event) since `analyzePassword()` is cheap pure computation — no reason to make someone click a button to see the meter move. That's also why it has extra `strengthLive` / `lastAnnouncedLabel` logic — to stop a screen reader from announcing the strength on literally every character typed, only when the tier actually changes.

Summary of the full path: HTML skeleton (built at compile time from JSON data) → user interaction in the browser → plain object gathered from form fields → one call into `src/lib/*.js` → plain result object → HTML string built (with `escapeHtml` on anything dynamic) → dropped into the page via `.innerHTML`.